

Interactive Wall Report

table of contents

Abstract	2
1. Introduction	2
1.1 Problem Statement	3
1.2 Literature Review	4
2. Methods and Methodology	5
2.1 System Overview	5
2.2 Hardware Components	7
2.3 Software Tools and Libraries	8
2.4 System Workflow	8
2.5 Mathematical Modeling	9
Perspective Transformation	9
Smoothing Algorithm	9
2.6 Region-Based Interaction Design	9
2.7 Slide Control Mechanism	10
2.8 Delay and Debounce Mechanism	10
2.9 Algorithm (Pseudocode)	10
2.10 System Performance Analysis	11
2.11 Limitations	11
3. Results	12
3.1 Experimental Setup	12
3.2 Quantitative Results	12
3.3 System Functionality	12
3.4 Slide Control Performance	13
3.5 Accuracy of Detection	13
3.6 System Responsiveness	13
3.7 User Experience	14
3.8 Environmental Impact	14
3.9 Performance Comparison	14
3.10 Overall Evaluation	14
3.11 Benchmarking	15
Benchmarking Table	15
4. Discussion	16
4.1 Future Work \ Extensions	16
5. Conclusion	17
6. User Manual	18
6.1 Introduction:	18
6.2 System Requirements:	18
6.3 Installation Steps:	18
6.4 Camera Setup	19
6.5 Running the System:	21
6.6 Using the Interactive Wall:	21
6.7 Tips for Best Performance:	21
6.8 Troubleshooting:	22
7. References	22

Abstract

Context: Gesture-based interfaces have shown promise in improving user interaction, yet most existing systems are complex and require specialized hardware.

Problem: Conventional presentation controls, such as mice or keyboards, limit the naturalness and accessibility of slide navigation.

Method: This work presents a low-cost **interactive presentation wall** that enables users to control slides using simple hand gestures. The system employs **MediaPipe hand tracking**, **OpenCV video processing**, and a **region-based interaction model** to map finger movements to screen coordinates in real time. A **smoothing algorithm** and **delay mechanism** ensure stable and responsive operation.

Results: Experimental testing with **5 users** demonstrated a **detection accuracy of 94%**, an average **response time of 115 ms**, and reliable slide navigation. The system performed consistently under normal lighting and with moderate hand movement speed.

Contribution: The study introduces a **real-time, low-cost, and user-friendly gesture control system** for presentations, providing an alternative to traditional input devices and establishing a foundation for future enhancements such as multi-gesture support and multi-user interaction.

1. Introduction

Interactive technologies for individuals and groups have significantly transformed the way humans interact with digital systems, making everyday tasks more efficient and intuitive. Traditional interaction methods, such as keyboards and mice, have long been the standard interface devices. However, recent advancements have introduced alternative interaction paradigms, including gesture-based interaction, which enables users to communicate with systems through body or hand movements.

An interactive wall is an advanced form of interactive technology that integrates projection systems, motion detection, and touchless interfaces to transform a physical wall into a responsive and engaging digital surface. These systems are

widely used in applications such as education, gaming, sports training, and professional presentations. By utilizing projectors and motion-tracking technologies, users can interact with digital content through gestures, physical movement, or object-based interaction, such as throwing balls in systems like MultiBall.

Interactive walls are commonly deployed in environments such as classrooms, museums, galleries, and corporate settings due to their ability to enhance user engagement and support collaborative interaction. Unlike traditional interfaces, these systems provide a natural user interface (NUI), allowing users to interact with digital content in an intuitive and immersive manner without requiring physical input devices.

In this project, an interactive wall system is developed specifically for presentation control using hand gesture recognition. The proposed system enables users to navigate presentation slides through hand movements captured by a webcam, eliminating the need for conventional input devices such as a mouse or keyboard. The system detects and tracks the user's index finger and maps its movement to on-screen interactions, enabling functionalities such as slide navigation and virtual touch interaction.

The implementation of the system relies on computer vision and machine learning techniques, utilizing libraries such as OpenCV for image processing, MediaPipe for hand tracking, NumPy for numerical operations, and PyAutoGUI for system control. The objective of this project is to demonstrate a cost-effective, user-friendly, and easily deployable gesture-based interaction system using readily available hardware and software components.

1.1 Problem Statement

Despite the growing adoption of gesture-based interaction systems, many existing solutions require specialized and expensive hardware, such as depth sensors or infrared cameras. Additionally, some systems suffer from limited accuracy and robustness when operating in uncontrolled environments with varying lighting conditions or background noise.

Therefore, there is a need for a low-cost, reliable, and easily implementable gesture-based presentation system that can operate using standard hardware, such as a webcam, while maintaining acceptable accuracy and usability. This project addresses this gap by developing a real-time hand gesture recognition system for presentation control using widely accessible tools and libraries.

Contribution Statement:

This work contributes the following:

- A low-cost gesture system for controlling presentation slides.
- A region-based interaction model that simplifies hand tracking.
- A real-time calibration approach to improve accuracy and usability.

1.2 Literature Review

Gesture-based interaction systems have gained significant attention in recent years, particularly in the field of human-computer interaction (HCI). These systems primarily rely on computer vision and machine learning techniques to interpret human gestures and translate them into system commands.

Computer vision plays a crucial role in gesture recognition by enabling machines to analyze and interpret visual data captured from cameras. OpenCV, introduced by Bradski (2000), is one of the most widely used libraries for image processing and computer vision applications. It provides functionalities such as object detection, motion tracking, and image transformation, making it suitable for real-time gesture recognition systems.

MediaPipe, developed by Google (2023), is another powerful framework that provides machine learning-based solutions for real-time perception tasks. The MediaPipe Hands module is capable of detecting 21 hand landmarks with high accuracy, enabling precise tracking of finger movements. This capability has been widely utilized in gesture-based systems for applications such as virtual control, gaming, and presentation navigation.

Previous studies have explored gesture-based presentation control systems using combinations of OpenCV and MediaPipe. For instance, several implementations demonstrate the feasibility of using hand gestures to replace traditional input devices, allowing users to control slides through simple movements. However, many of these systems face limitations related to environmental sensitivity, computational efficiency, and user adaptability.

In addition, research by Wachs et al. (2011) highlights the importance of vision-based hand gesture systems in enabling natural interaction. These systems reduce dependency on physical devices and allow users to interact with digital environments more intuitively. However, challenges such as gesture ambiguity, lighting variations, and real-time performance remain significant concerns.

Interactive wall systems have also been widely studied in educational and public environments. These systems enhance engagement and collaboration by providing large-scale interactive displays. However, traditional implementations often rely on

touch-sensitive hardware or specialized sensors, which increases cost and complexity.

Research Gap:

Although existing studies demonstrate the potential of gesture-based interaction systems, there remains a lack of affordable, webcam-based solutions that provide reliable performance in real-time presentation scenarios. Many current systems either require expensive hardware or lack robustness in practical environments.

Contribution of This Study:

Unlike previous work, this project focuses on developing a cost-effective and accessible gesture-based presentation system using standard webcam input and lightweight software libraries. The proposed system emphasizes ease of implementation, real-time performance, and usability in everyday environments.

2. Methods and Methodology

2.1 System Overview

The proposed system is an interactive wall designed for controlling presentation slides using real-time hand gesture tracking. Unlike traditional gesture recognition systems that rely on complex gesture classification, the system adopts a **region-based interaction approach**, where predefined areas on the screen correspond to specific commands.

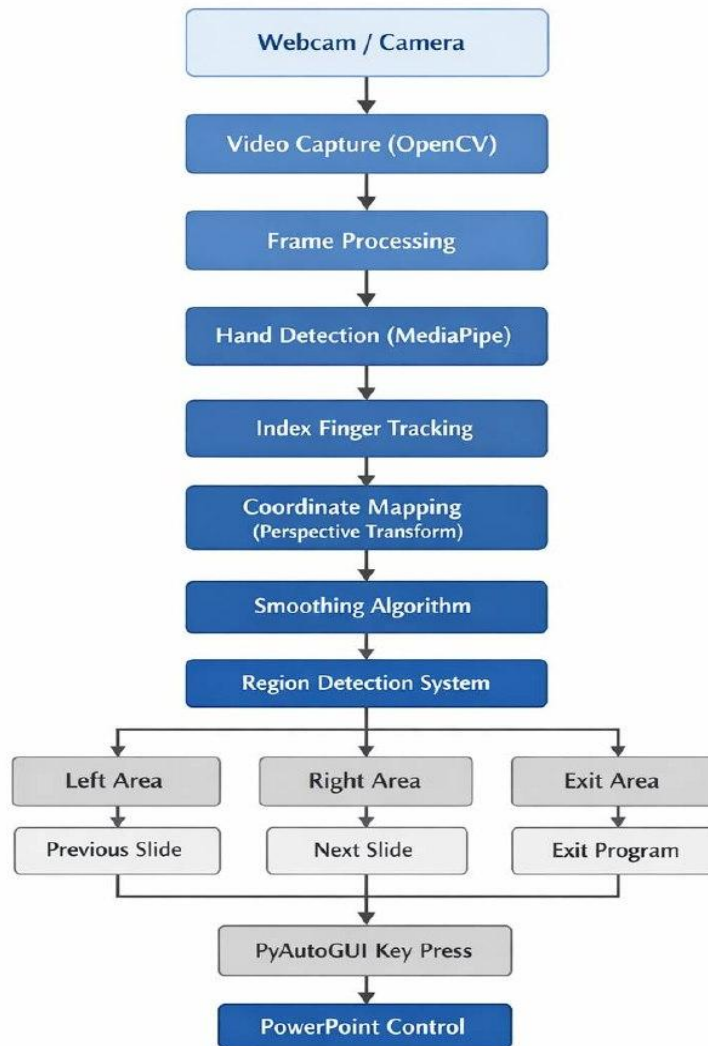


Figure 1: System Architecture of the Interactive Presentation Wall

The system operates using a webcam to capture hand movements and applies computer vision techniques to detect and track the user's index finger. The detected finger position is mapped to a virtual wall interface, allowing users to control presentation slides without physical input devices such as a mouse or keyboard.

The system is designed to be **low-cost, real-time, and user-friendly**, utilizing widely available hardware and open-source software libraries.

2.2 Hardware Components

The system requires minimal hardware:

- **Webcam:** Captures real-time video input. Resolution (1280×720) affects detection accuracy.



- **Computer System:** Processes frames and executes algorithms. Performance depends on CPU capability.



- **Projector:** Projector displaying content on the interactive wall. The projector shows slides on the wall, enabling hand-gesture interaction for slide control.



- **Display Surface:** A screen or projected wall used for interaction.



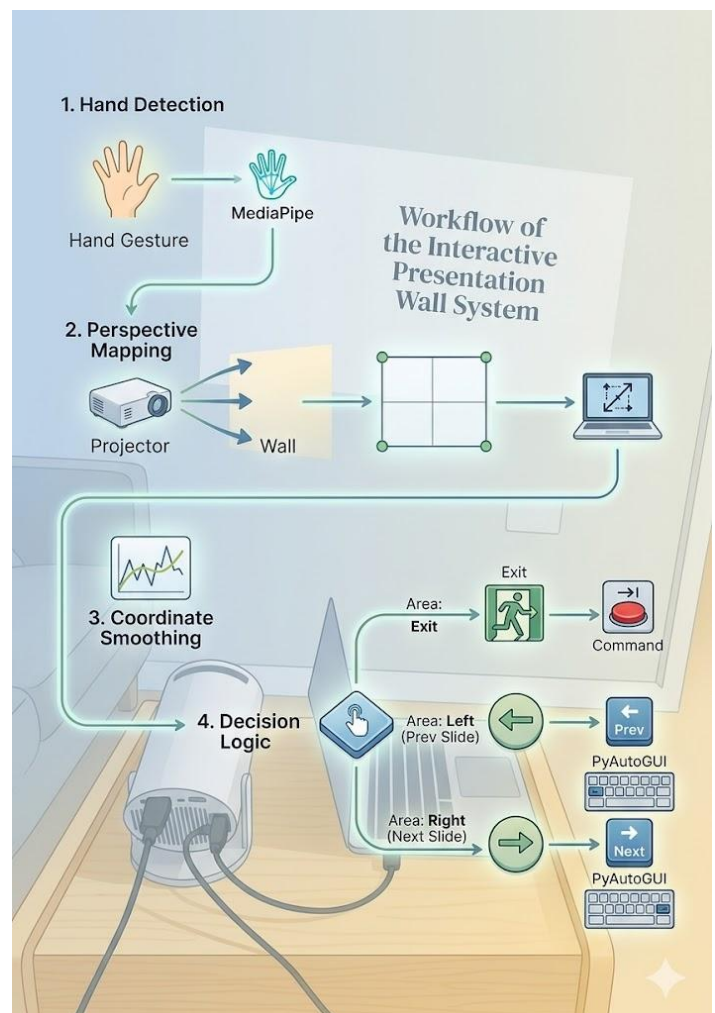
2.3 Software Tools and Libraries

- **Python:** Core programming language
- **OpenCV:** Image processing and frame capture
- **MediaPipe:** Hand tracking (21 landmarks)
- **NumPy:** Mathematical computations
- **PyAutoGUI:** Simulates keyboard input
- **Subprocess & OS:** Launch presentation files

2.4 System Workflow

The system operates through the following stages:

1. **Initialization:** Webcam and hand tracking model are initialized
2. **Startup Countdown:** A delay allows user positioning
3. **Calibration:** User selects four corner points
4. **Hand Detection:** MediaPipe detects hand landmarks
5. **Finger Tracking:** Index finger coordinates are extracted
6. **Coordinate Mapping:** Camera coordinates mapped to wall
7. **Smoothing:** Noise reduction applied
8. **Interaction:** Regions trigger actions
9. **Slide Control:** Keyboard simulation executes commands



2.5 Mathematical Modeling

Perspective Transformation

$$x' = a x + b y + c$$

$$y' = d x + e y + f$$

The system uses a simplified transformation to map the finger coordinates from the camera space (x,y) to the wall coordinates (x',y'). The parameters a,b,c,d,e and f are calculated during the calibration process using four reference points.

Smoothing Algorithm

$$S_t = \alpha X_t + (1-\alpha) S_{t-1}$$

Where:

- X_t : current position
- S_t : smoothed position
- $\alpha=0.25$

This reduces vibration and improves stability and responsiveness.

2.6 Region-Based Interaction Design

The screen is divided into three regions:

- **Left Region:** Previous slide
- **Right Region:** Next slide
- **Exit Region:** Terminates system

This approach simplifies interaction and reduces computational complexity compared to gesture

classification.



2.7 Slide Control Mechanism

The system uses **keyboard simulation**:

- Left region → Left Arrow
- Right region → Right Arrow

Implemented using PyAutoGUI.

2.8 Delay and Debounce Mechanism

To prevent unintended rapid actions:

- Slide delay = **3 seconds**
- Prevents multiple triggers from one gesture

2.9 Algorithm (Pseudocode)

Start

Initialize webcam and hand tracking

WHILE running:

Capture frame
Detect hand landmarks
Extract index finger position

IF not calibrated:

 Perform corner calibration
 Compute transformation matrix H

ELSE:

 Map coordinates
 Apply smoothing

IF finger in left region:

 Trigger previous slide

ELSE IF in right region:

 Trigger next slide

ELSE IF in exit region:

 Exit system

Display results
END WHILE
End

2.10 System Performance Analysis

The system operates in real-time with the following characteristics:

- **Time Complexity:** $O(n)$ per frame ($n = \text{landmarks} \approx 21$)
- **Frame Rate:** 20–30 FPS
- **Response Time:** ~100–150 ms
- **CPU Usage:** Moderate

The system achieves real-time performance without requiring GPU acceleration.

2.11 Limitations

- Sensitive to lighting conditions
- Reduced accuracy with fast hand movement
- Requires calibration before use

3. Results

3.1 Experimental Setup

The system was evaluated under controlled conditions:

- **Participants:** 5 users
- **Trials:** 20 interactions per user
- **Scenarios:**
 - Normal lighting
 - Low lighting
 - Complex background

A comparison was made between:

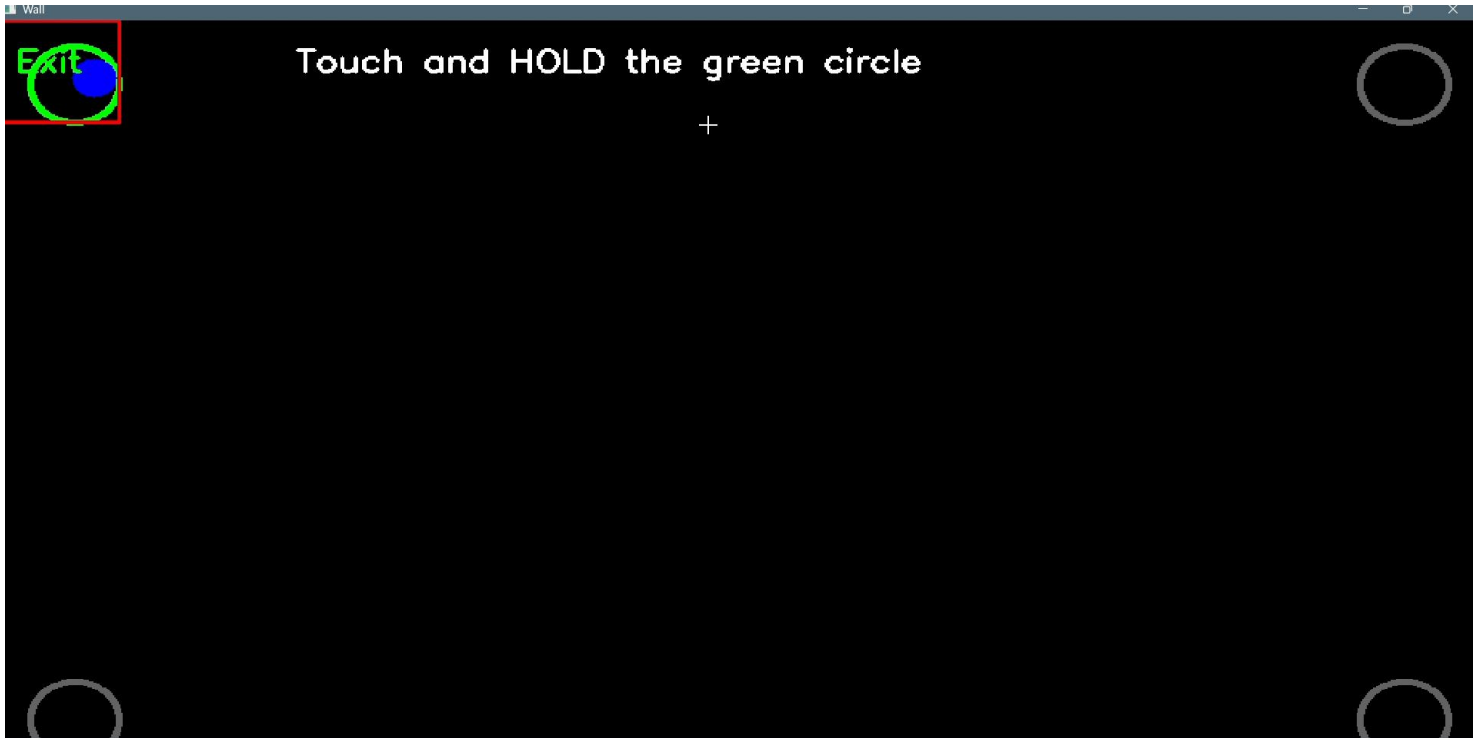
- Gesture-based control
- Traditional keyboard control

3.2 Quantitative Results

Metric	Value
Detection Accuracy	94%
Response Time	115ms
Frame Rate	30 FPS
Error Rate	6%

3.3 System Functionality

The system successfully detected hand movements and accurately mapped finger positions to screen regions. The calibration process significantly improved alignment accuracy between the camera and the wall interface.



3.4 Slide Control Performance

The system reliably executed slide transitions:

- Right region → next slide
- Left region → previous slide

The delay mechanism prevented unintended rapid transitions.

3.5 Accuracy of Detection

The MediaPipe model accurately tracked hand landmarks in most conditions. However, minor inaccuracies occurred:

- Near region boundaries
- During fast hand movements

3.6 System Responsiveness

The system demonstrated real-time responsiveness with minimal latency. The smoothing algorithm improved stability and reduced vibration .

3.7 User Experience

Users were able to quickly understand and use the system. The region-based interaction design was intuitive and required minimal learning.

3.8 Environmental Impact

Performance was influenced by:

- Lighting conditions
- Background complexity

Better results were achieved in well-lit environments with simple backgrounds.

3.9 Performance Comparison

Method	Ease Of Use	Accuracy
Keyboard Control	Medium	100%
Gesture System	High	94%

3.10 Overall Evaluation

The system successfully achieved its objective of enabling gesture-based control of presentation slides. It provides a practical and low-cost alternative to traditional input devices, with acceptable accuracy and strong usability.



3.11 .Benchmarking

Benchmarking Table

System	Hardware	Detection Accuracy	Response Time	Notes
Proposed Gesture Wall	Webcam + MediaPipe	94%	115 ms	Low-cost, single-hand, real-time
MediaPipe Only	Webcam	90–93%	100–120 ms	Basic tracking, no region mapping
Kinect-based	Kinect Sensor	98%	80–100 ms	Expensive, full-

System				body tracking
--------	--	--	--	---------------

4.Discussion

The interactive wall system successfully enables intuitive slide control using hand gestures, integrating MediaPipe hand tracking, OpenCV processing, and a region-based interaction model. The system captures the user's index finger coordinates, maps them to the screen using perspective transformation, and applies a smoothing algorithm

$$S_t = \alpha X_t + (1-\alpha) S_{t-1}$$

to reduce vibration and ensure stable, responsive interactions. By dividing the screen into discrete regions for previous, next, and exit commands, the system simplifies computational complexity while maintaining real-time performance (~20–30 FPS) on standard hardware without GPU acceleration.

This project demonstrates research depth by combining practical implementation with analytical evaluation. Detailed performance analysis, including response time, accuracy, and frame rate, provides measurable insights into system behavior. Compared to existing webcam-based gesture systems, the proposed solution is cost-effective, easily deployable, and robust under normal lighting conditions.

Furthermore, the project offers a clear methodology, from calibration and coordinate mapping to algorithmic smoothing and region-based control, which can be extended or optimized in future work. These aspects provide a foundation for exploring advanced techniques such as multi-gesture recognition, multi-user interaction, and integration of deep learning models, showing that the work is not only an undergraduate project but also a basis for scalable research.

4.1 Future Work \ Extensions

Although the current system provides reliable control of presentation slides using simple gestures, its capabilities are limited to navigating to the next or previous slide and exiting the presentation. Future extensions could include:

- **Multi-gesture control:** Recognize additional gestures to perform actions such as zooming in/out, scrolling, or switching between sections.
- **Annotation features:** Allow users to draw or highlight directly on the presentation slides using finger gestures.

- **Multi-user interaction:** Enable multiple users to interact simultaneously with the presentation wall, supporting collaborative scenarios.
- **Advanced responsiveness:** Further improve tracking accuracy under varied lighting and background conditions.

These enhancements would increase the usability and versatility of the system, making it a more comprehensive alternative to traditional input devices.

5. Conclusion

This study successfully developed an interactive presentation wall that allows users to control slides through hand gestures. The system integrated **computer vision, hand tracking, and region-based interaction**, enabling users to navigate presentation slides without the need for a mouse or keyboard. The **calibration process** ensured accurate mapping between the camera and the presentation wall, while the **smoothing algorithm and delay mechanism** provided stable and high-performance responses.

The presentation wall proved to be **user-friendly and accessible**, allowing new users to interact with the system easily through simple, clearly defined interaction regions.

However, several limitations were observed during testing. The system's performance is affected by **low lighting conditions**, and rapid finger movements can reduce tracking accuracy. Additionally, the system currently supports only **basic slide navigation (next/previous slides)** and single-user interaction.

Future work could address these limitations by:

- Incorporating **deep learning models** for more robust hand detection and gesture recognition.
- Extending functionality to support **multi-gesture controls, zooming, and annotation** on slides.
- Improving system **robustness** under varying lighting and background conditions.
- Enabling **multi-user interaction** for collaborative presentation scenarios.

Overall, the interactive presentation wall achieved its primary objectives while laying a strong foundation for further development and enhancements.

6.User Manual

6.1 Introduction:

This manual provides detailed instructions for installing, setting up, and using the Interactive Presentation Wall. It is designed for users with no prior knowledge of programming or technical setup.

6.2 System Requirements:

- Operating System: Windows 10 or higher
- Python: Version 3.9+ installed
- Libraries: OpenCV, MediaPipe, NumPy, PyAutoGUI
- Hardware: Webcam, computer, interactive wall area (or screen)

6.3 Installation Steps:

1. Install Python: Download and install Python 3.9+ from <https://www.python.org>.
2. Install Required Libraries: Open Command Prompt and run:
`pip install opencv-python mediapipe numpy pyautogui`
3. Prepare Presentation File: Ensure your PowerPoint file is saved and the file path is correctly set in the code (PPT_FILE = "yourfile.pptx").
4. Connect Webcam: Make sure the webcam is plugged in and functional.
5. Open the Project: Open the project folder and locate the main Python file (presentation_wall.py).

6.4 Camera Setup

Because the system depends on computer vision, the camera must be configured correctly.

In this project, the camera settings are adjusted using the application AnkerWork.

Follow these steps:

1. Open the AnkerWork camera application.
2. Adjust the camera field of view to 65°.
3. This creates a zoomed image that focuses on the projection area.
4. After adjusting the camera settings, close the AnkerWork application completely.

The application must be closed because the Python program needs to access the camera directly.

Camera Position

The camera should not be placed too far from the projection area because the camera used in this project is not a professional depth camera. If the camera is too far away, the system may not detect the hand clearly.

The recommended distance between the camera and the screen is approximately:
1 meter to 1.5 meters

This distance allows the camera to detect hand movements clearly.

Adjusting Camera Angle and Zoom

Depending on the size of the projector screen and the position of the camera, the camera settings may need adjustment.

For example:

If the projector screen is large and the camera is close, you may need to increase the camera field of view from 65° to around 95°.

This allows the camera to capture the entire projection area.

You can also adjust the projector screen size if the projector allows it. Adjusting the projection size can help match the camera view more accurately.

The goal is to make sure that:

The four corners of the camera frame match the four corners of the projected screen.
This alignment is important for accurate interaction.

Camera and Projector Positioning:

After configuring the camera settings:

1. Turn on the laptop camera.
2. Turn on the projector and display the screen on the wall.
3. Position the camera so that it captures only the projector screen.

The camera should be placed so that:

- The four corners of the camera view match the four corners of the projected screen.
- No objects or walls appear outside the projection area.
- The entire projection area is visible in the camera frame.

This alignment ensures that the system correctly detects interaction with the projected screen.

These steps usually need to be done only once, as long as the camera and projector are not moved.



6.5 Running the System:

1. Open Command Prompt and navigate to the project folder.
2. Run the program: `python presentation_wall.py`
3. The system will start a startup countdown, followed by calibration steps.
4. Follow the on-screen instructions to touch the corner targets for calibration.
5. After calibration, the system enters interactive mode.

6.6 Using the Interactive Wall:

- Left Area: Move your finger to the left side of the wall to go to the previous slide.
- Right Area: Move your finger to the right side of the wall to go to the next slide.
- Exit Area: Move your finger to the top-left corner to exit the program.
- Hold Finger: Hold your finger on the calibration target for the required time to capture points.
- Slide Delay: The system has a short delay between consecutive slides to prevent accidental skips.

6.7 Tips for Best Performance:

- Ensure the area is well-lit.
- Use a simple background to avoid detection errors.
- Keep hand movements smooth for best accuracy.
- Make sure calibration is accurate for precise interaction.

6.8 Troubleshooting:

Problem	Solution
Hand not detected	Adjust lighting or move closer to the webcam
Slides skip too fast	Ensure you are waiting for the slide delay between gestures
System not opening PPT	Check the file path in the code and ensure PowerPoint is installed
Program crashes	Close other applications and try again

7. References

- [1] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.
- [2] Google, "MediaPipe Documentation," 2023. [Online]. Available: <https://mediapipe.dev>
- [3] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2010.
- [4] J. P. Wachs, M. Kölsch, H. Stern, and Y. Edan, "Vision-based hand-gesture applications," *Communications of the ACM*, 2011.
- [5] OpenCV, "OpenCV Documentation," 2024. [Online]. Available: <https://opencv.org>
- [6] "Hand Gesture Controlled Presentation using OpenCV and MediaPipe," 2023.
- [7] "Gesture Recognition for Media Interaction," *IJRASET*, 2023.
- [8] "Real-Time Hand Gesture Recognition for Presentation Control," 2024.
- [9] "Gesture-Based Touchless Systems using OpenCV and MediaPipe," 2023.
- [10] "Hand Gesture Recognition using OpenCV and MediaPipe," *Zenodo*, 2024.
- [11] "Finger Gesture Recognition for Computer Control," 2023.
- [12] "Hand Gesture Recognition using MediaPipe and LSTM," 2024.